

Exceptions for system level services

Page No.	
Date	

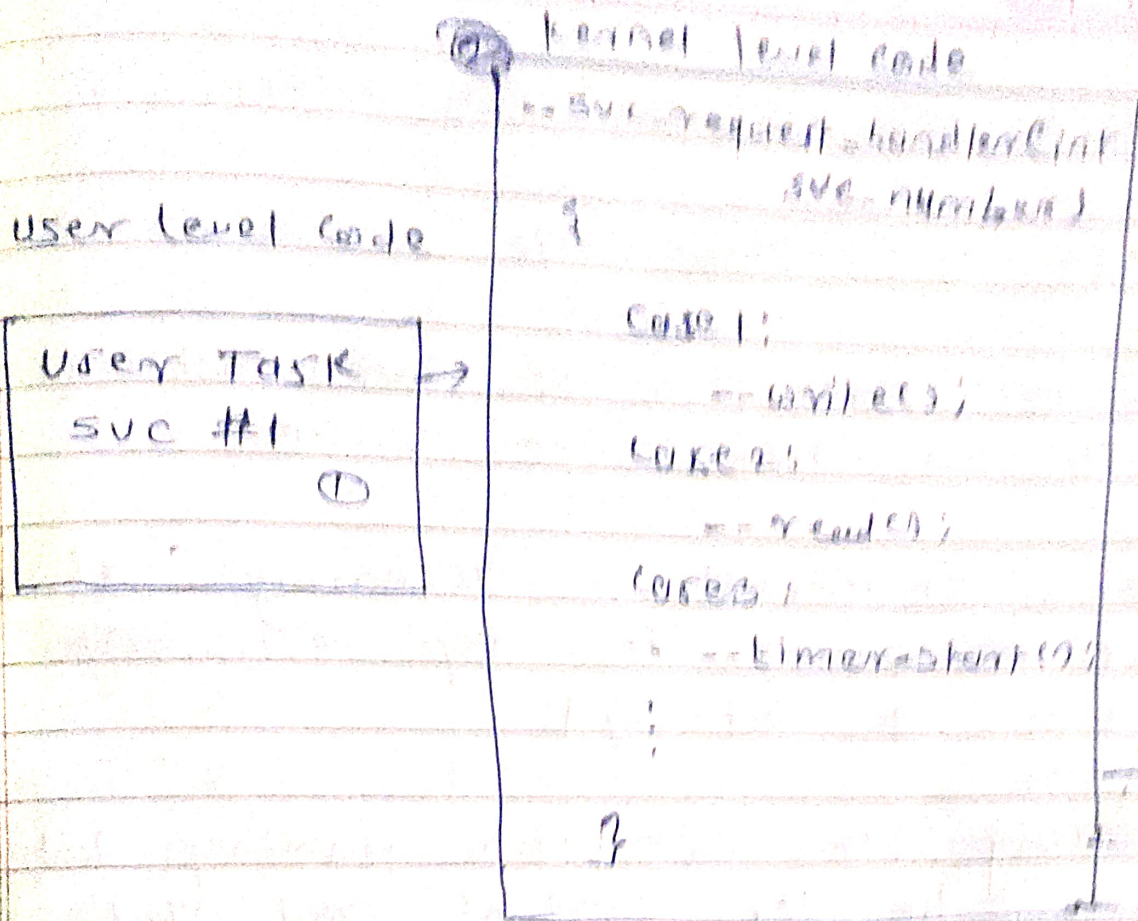
- ARM Cortex Mx processor supports 2 important system level services exception.

SVC (Supervisor Call) &
PendSV (Pendable Service)

- Supervisory calls are typically used to request privileged operations or access to system resources from an operating system.
- SVC exception is mainly used in an OS environments, for example, A less ~~pre~~ privileged user task can trigger SVC exception to get system-level services (like accessing device drivers, peripherals from the kernel of the OS).
- PendSV is mainly used in an OS environments to carry out context switching between two or more tasks when no other exceptions are active in the system.

• SVC (Supervisor Call) Instruction

- SVC is a thumb ISA instruction which causes SVC exception.
- In an RTOS scenario, user tasks can execute SVC instruction with an associated argument to make a supervisory call to seek privileged resources from the kernel code.
- Unprivileged user tasks use the SVC instructions to ~~changed~~ change the processor mode to privileged mode to access privileged resources like peripherals.
- SVC instruction is always used along with number, which can be used to identify the request type by the kernel code.
- The SVC handler executes right after the SVC instruction (no delay, unless a higher priority exception arrives at the same time).



privileged resource
(display)

peripheral register
access is protected
by MPU
(privileged access
only)

- Methods to trigger svc exceptions

- There are two ways

- ① Direct execution of svc instruction with an immediate value

ex. 'svc #0x04' in assembly (using svc instruction is very efficient in terms of latency)

- ② Setting the exception pending bit in "System handler control and state register (uncommon method)

★ How to extract the svc number:

- The svc instruction has a number embedded within it, often referred to as the svc number.

- In the svc handler, you should fetch the opcode of the svc instruction and then extract the svc number.

- To fetch the opcode of svc instruction from program memory, we should have the value of PC (return address) where the user code had interrupted while triggering the svc exception.

Page No.
 Date

- The value of the pc (return address) where the user code had interrupted is stored in the stack as part of exception entry sequence by the processor.

→ PendSV Exception :

- It is an exception type 14 and has a programmable priority level.

- This exception is triggered by setting its pending status by writing to the "interrupt control and state register" of processor.

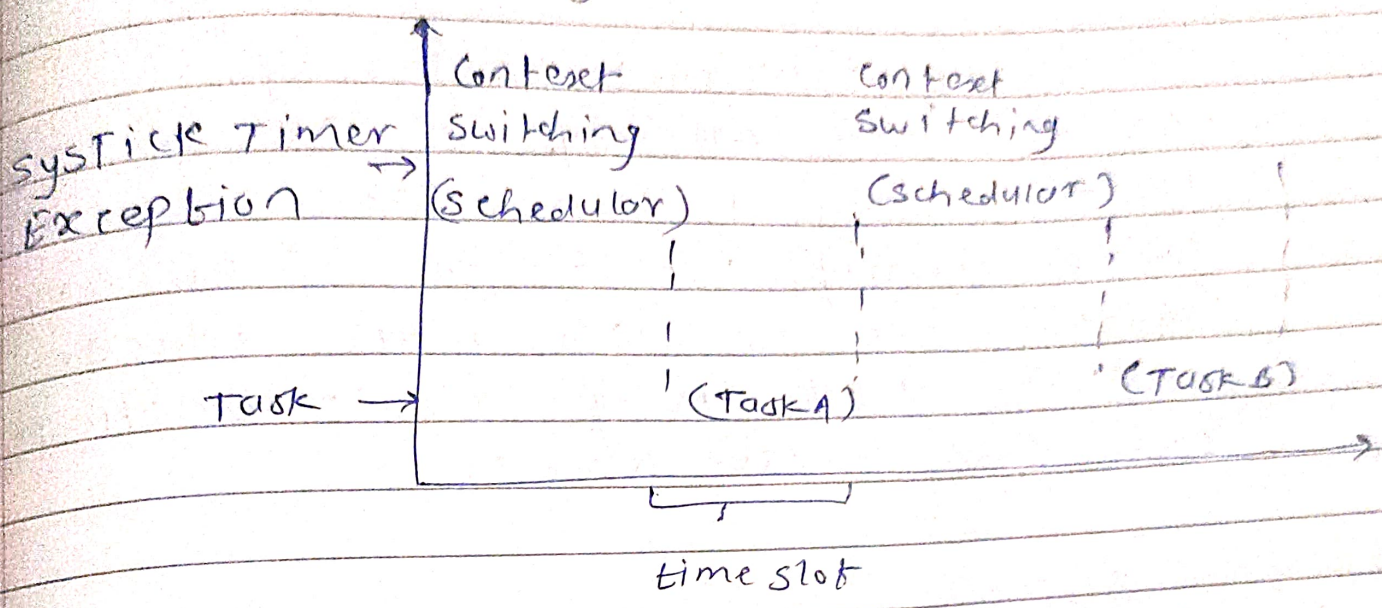
- Triggering a pendsv system exception is way of invoking the pre-emptive kernel to carry out the context switch in an os environment.

- In an os environment, pendsv handler is set to the lowest priority level and the pendsv handler carries out the context switch operation.

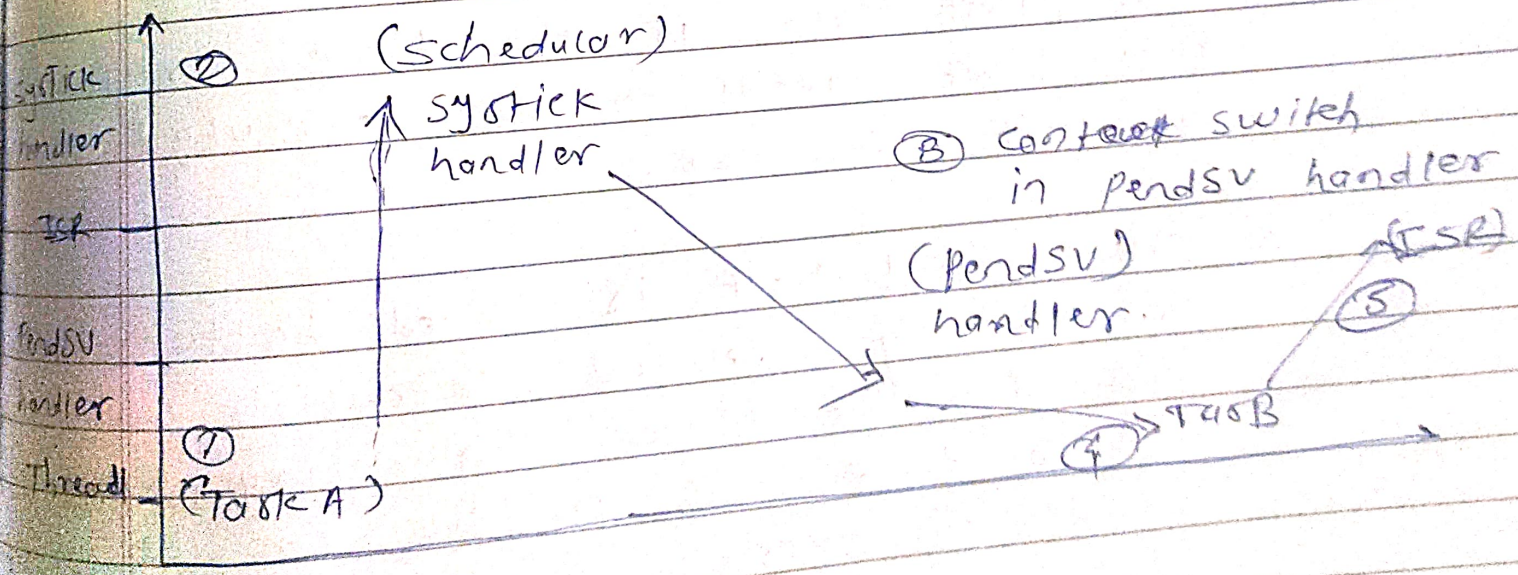
• Typical use of PendSV

- Typically this exception is triggered inside a higher priority exception handler and it gets executed when the higher priority handler finishes.
- Using this characteristic, we can ~~the~~ schedule the pendSV exception handler to be executed after all the other interrupt processing tasks ~~and~~ are done.
- This is very useful for a context switching operation, which is crucial operation in various OS designs.
- Using PendSV in context switching will be more efficient in an interrupt noisy environment.
- In an interrupt noisy environment and we need to delay the context switching until all IRQs are executed.

Context Switching



PendSV cont in context switching



Page No.
 Date

* pendSV other use cases:

- offloading interrupt processing
- If a higher priority handler is doing time consuming work, then the other lower priority interrupts will suffer, and system responsiveness may reduce. This can be solved using a combination of ISR and pendSV handler.

* Offloading ~~in~~ interrupt processing using pendSV.

- Interrupt may be serviced in 2 halves

1. The first half is the time critical part that needs to be executed as a part of ISR.
2. The second half is called bottom half is basically delayed execution where rest of the time consuming work will be done. So, pendSV can be used in these cases, to handle the second half execution by triggering it in the first half.